EuroSAT CW1 - Cell-by-Cell Demonstration Script

Simple plain English | Short format | Complete guide for all 8 notebooks

Notebook 01: Data Download (01_data_download.ipynb)

Cell 1: Setup & Clone

What it does: Clones the GitHub repository and sets the working folder.

Say: "In Cell 1, I set up the environment and clone my coursework repository from GitHub."

Cell 2: Mount Drive

What it does: Connects Google Drive for persistent storage.

Say: "In Cell 2, I mount Google Drive. This ensures that all downloaded data, models, and metrics are permanently saved."

Cell 3: Download Dataset

What it does: Downloads the 27,000 satellite images using torchvision.

Say: "In Cell 3, I download the official EuroSAT RGB dataset. It contains 27,000 satellite image patches across 10 land-use classes."

Cell 4: Verify Folders

What it does: Checks that all 10 class folders exist and counts images.

Say: "In Cell 4, I verify the download. Each class has between 2,000 and 3,000 images, and all images are 64 by 64 pixels."

Cell 5: Copy to Drive

What it does: Backs up the images to Google Drive.

Say: "In Cell 5, I back up the raw images to Google Drive. This avoids downloading them again in future sessions."

Cell 6: Save Metadata

What it does: Records dataset source, date, citation, and class names.

Say: "In Cell 6, I save the dataset metadata, citing Helber et al. (2019) and recording Sentinel-2 sensor details."

Notebook 02: EDA & Leakage-Safe Split (02_eda_split.ipynb)

Cell 1: Setup

What it does: Imports libraries for hashing, plotting, and splitting.

Say: "In Cell 1, I import NumPy, PIL, and hashing tools for data auditing."

Cell 2: Check Hashes

What it does: Verifies image integrity and generates MD5 hashes.

Say: "In Cell 2, I check every image with PIL and calculate MD5 hashes to detect corrupt files or duplicates."

Cell 3: Build Array

What it does: Packages all 27,000 images into a single eurosat_64.npz archive.

Say: "In Cell 3, I pack the images and labels into a single NumPy array on Google Drive for fast local loading."

Cell 4: Audit Report

What it does: Prints the audit report.

Say: "In Cell 4, you can see the audit report. There are zero corrupt images and zero exact duplicates."

Cell 5: Class Bar Chart

What it does: Generates a bar chart of class counts.

Say: "In Cell 5, I plot the class distribution. Pasture has 2,000 images while Forest has 3,000. This mild imbalance justifies using Macro-F1."

Cell 6: Sample Grid

What it does: Displays 3 sample images for each of the 10 classes.

Say: "In Cell 6, I display sample images. This shows visual similarities, like green pastures looking very close to herbaceous vegetation."

Cell 7: Pixel Histograms

What it does: Plots red, green, and blue intensity distributions.

Say: "In Cell 7, I plot channel histograms. The three color channels have different distributions, which justifies per-channel normalization."

Cell 8: Stratified Split

What it does: Splits indices into 70% train, 15% validation, and 15% test.

Say: "In Cell 8, I split the dataset into 70% train, 15% validation, and 15% test. It is stratified to keep class balance identical across splits."

Cell 9: Leakage Check

What it does: Verifies zero hash overlap between splits.

Say: "In Cell 9, I run a leakage check. There are zero hash overlaps between train, val, and test."

Cell 10: Normalization

What it does: Calculates mean and std strictly on train and saves split indices.

Say: "In Cell 10, I calculate channel mean and std strictly on the training set. This guarantees zero data leakage."

Notebook 03: NumPy MLP from Scratch (03_numpy_mlp.ipynb)

Cell 1: Setup & Load

What it does: Loads saved dataset and split indices.

Say: "In Cell 1, I load the saved dataset and training indices."

Cell 2: 4-Class Subset

What it does: Extracts 4 classes downsampled to 16x16 grayscale (256 features).

Say: "In Cell 2, I prepare the Task 2 teaching dataset. I take 4 classes and downsample them to 16 by 16 grayscale, giving 256 inputs."

Cell 3: NumPy MLP Class

What it does: Hand-coded forward and backward propagation in pure NumPy.

Say: "In Cell 3, I implement the neural network from scratch using only NumPy. It uses a tanh hidden layer and Softmax output. The backward pass derives exact gradients using the chain rule."

Cell 4: Train MLP

What it does: Trains using gradient descent for 200 epochs.

Say: "In Cell 4, I train the network. The initial loss is 1.44, very close to the theoretical value of ln(4) = 1.386. The final loss drops to 0.50."

Cell 5: Loss Curve Plot

What it does: Plots epoch vs cross-entropy loss.

Say: "In Cell 5, I plot the training curve. It shows smooth, stable convergence."

Cell 6: Gradient Check

What it does: Compares analytical gradients against central differences (eps=1e-5).

Say: "In Cell 6, I verify the backprop math using a numerical gradient check. All 12 parameters have relative errors below 1e-10. This proves the hand-coded gradients are 100% correct."

Cell 7: Overfit Tiny Batch

What it does: Trains on 10 images to confirm loss approaches zero.

Say: "In Cell 7, I overfit a tiny batch of 10 samples. The loss drops to 0.001, confirming the model has no implementation bugs."

Notebook 04: Baseline Model (04_baseline_hog_lr.ipynb)

Cell 1: Setup

What it does: Imports scikit-learn, skimage, and loading tools.

Say: "In Cell 1, I set up the environment for the classical machine learning baseline."

Cell 2: Feature Extractor

What it does: Extracts HOG edge features and color histograms (1,788 features).

Say: "In Cell 2, I extract hand-crafted features: HOG captures edge orientation, and color histograms capture color patterns."

Cell 3: Scale Features

What it does: Fits StandardScaler strictly on training set.

Say: "In Cell 3, I scale the features. The scaler is fit strictly on training features to prevent leakage."

Cell 4: Hyperparameter Tuning

What it does: Evaluates 6 values of C on validation Macro-F1.

Say: "In Cell 4, I tune the regularization parameter C over 6 trials. C = 0.01 achieves the highest validation Macro-F1 of 0.7299."

Cell 5: Train Final Model

What it does: Trains final Logistic Regression with C = 0.01.

Say: "In Cell 5, I train the final baseline model with the best C value and save the checkpoint."

Cell 6: Validation Score

What it does: Evaluates precision, recall, and F1 on validation data.

Say: "In Cell 6, I evaluate validation performance. Macro-F1 is 0.7299 and accuracy is 73.6%."

Cell 7: Confusion Matrix

What it does: Plots validation confusion matrix.

Say: "In Cell 7, I plot the confusion matrix. Forest scores 94% F1, while Herbaceous Vegetation is harder at 55%."

Cell 8: Test Set Evaluation

What it does: Unfreezes and evaluates the final test set.

Say: "In Cell 8, I evaluate the unseen test set. The Baseline reaches 0.7324 Macro-F1 with 17,880 parameters."

Notebook 05: PyTorch MLP (05_mlp_pytorch.ipynb)

Cell 1: Setup

What it does: Initializes PyTorch and verifies GPU availability.

Say: "In Cell 1, I set up PyTorch and enable the GPU."

Cell 2: PyTorch Dataset

What it does: Loads image tensors with train-only normalization and augmentation.

Say: "In Cell 2, I build the PyTorch Dataset with data augmentation: horizontal flips, vertical flips, and rotations."

Cell 3: MLP Architecture

What it does: Defines 2 hidden layers (512 and 256 units) with ReLU and Dropout(0.3).

Say: "In Cell 3, I define the deep MLP. Because it flattens 64 by 64 RGB images, it requires over 6.4 million parameters."

Cell 4: Training Loop

What it does: Implements training and validation with early stopping.

Say: "In Cell 4, I implement training with Adam and early stopping with a patience of 5 epochs."

Cell 5: Train 3 Seeds

What it does: Trains separate models for seeds 0, 1, and 2.

Say: "In Cell 5, I train the MLP across 3 random seeds to evaluate variance."

Cell 6: Results Summary

What it does: Calculates mean and standard deviation across seeds.

Say: "In Cell 6, I summarize the results. The MLP achieves a Test Macro-F1 of only 0.5100 with 53% accuracy."

Cell 7: Loss Curves Plot

What it does: Plots train vs validation loss curves for all seeds.

Say: "In Cell 7, I plot the loss curves. The validation loss quickly diverges, showing that the flat MLP overfits heavily."

Cell 8: Test Confusion Matrix

What it does: Plots test confusion matrix for the best seed.

Say: "In Cell 8, I plot the confusion matrix. The MLP struggles because flat layers destroy the 2D spatial arrangement of images."

Notebook 06: PyTorch CNN (06_cnn_pytorch.ipynb)

Cell 1: Setup

What it does: Imports PyTorch neural network modules.

Say: "In Cell 1, I initialize PyTorch for the Convolutional Neural Network."

Cell 2: Dataset & Loaders

What it does: Builds batches of 64 images for GPU training.

Say: "In Cell 2, I initialize the DataLoader with batch size 64 and augmentation."

Cell 3: CNN Architecture

What it does: Defines 3 conv blocks (32, 64, 128) with BatchNorm, ReLU, and MaxPool.

Say: "In Cell 3, I build the CNN architecture. It has 3 convolutional blocks, batch normalization, and adaptive pooling. It uses only 94,986 parameters."

Cell 4: Training Setup

What it does: Defines Adam optimizer (lr=1e-3) and early stopping.

Say: "In Cell 4, I set up Adam optimizer with learning rate 1e-3 and early stopping."

Cell 5: Train 3 Seeds

What it does: Trains the CNN for seeds 0, 1, and 2, saving checkpoints.

Say: "In Cell 5, I train the CNN across 3 seeds. Notice how training is smooth and reaches over 90% validation F1."

Cell 6: Results Summary

What it does: Computes mean and standard deviation across seeds.

Say: "In Cell 6, I display the results. The CNN achieves a Test Macro-F1 of 0.9103 and 93% accuracy. It beats the MLP by 40% with 68 times fewer parameters."

Cell 7: Loss & Accuracy Plots

What it does: Plots training vs validation curves.

Say: "In Cell 7, I plot the curves. Training and validation lines track closely together, proving strong generalization."

Cell 8: Confusion Matrix

What it does: Plots final test set confusion matrix.

Say: "In Cell 8, I plot the confusion matrix. Classes like Forest, SeaLake, and Residential reach 97% to 98% F1."

Cell 9: Ablation Study

What it does: Retrains the CNN with dropout = 0.0.

Say: "In Cell 9, I run the ablation study by removing dropout. Validation F1 changes by only +0.0073, showing batch norm and augmentation already provide strong regularization."

Notebook 07: Experiments & Evaluation (07_experiments_evaluation.ipynb)

Cell 1: Setup & Restore

What it does: Restores all checkpoints, splits, and metrics from Drive.

Say: "In Cell 1, I load all saved checkpoints and test splits from Google Drive."

Cell 2: Master Summary Table

What it does: Compares Baseline, MLP, and CNN.

Say: "In Cell 2, I compare all models. CNN is the clear winner with 0.9103 F1, compared to 0.7324 for Baseline and 0.5100 for MLP."

Cell 3: Per-Class Report

What it does: Computes precision, recall, and F1 for all 10 classes.

Say: "In Cell 3, I display the per-class report. Overall test accuracy is 93%, with precision and recall above 90% across most classes."

Cell 4: Error Analysis

What it does: Identifies 288 misclassified images (7.1%) and shows 5 examples.

Say: "In Cell 4, I inspect model errors. Only 288 out of 4,050 images were misclassified. Errors mostly happen between visually similar classes like River and Highway."

Cell 5: Gaussian Noise Stress Test

What it does: Tests CNN under noise levels (0.0 to 0.20).

Say: "In Cell 5, I run the stress test. As noise increases to 0.20, Macro-F1 drops from 0.92 to 0.16. This reveals that the CNN is sensitive to atmospheric sensor noise."

Cell 6: Save Master CSV

What it does: Writes metrics/results.csv logging all models and seeds.

Say: "In Cell 6, I save the complete results into results.csv for reproducibility."

Notebook 08: Recommendation & Inference Tool (08_recommendation_inference.ipynb)

Cell 1: Setup

What it does: Loads class labels and train normalization stats.

Say: "In Cell 1, I load the class names and normalization parameters for the inference tool."

Cell 2: Recommendation Table

What it does: Compares accuracy, parameters, and trade-offs.

Say: "In Cell 2, I present the recommendation table. CNN is recommended because it combines the highest accuracy with a compact 94K parameter footprint."

Cell 3: Inference Latency

What it does: Benchmarks GPU inference time for batch of 64.

Say: "In Cell 3, I benchmark inference speed. The CNN processes a batch of 64 in only 0.73 milliseconds, which is fast enough for real-time satellite processing."

Cell 4: The predict() Function

What it does: Builds complete inference pipeline with thresholding.

Say: "In Cell 4, I define the production predict function. It checks file validity, normalizes the input, and returns class predictions with confidence scores."

Cell 5: Demo - Normal Satellite Image

What it does: Predicts on a valid test image.

Say: "In Cell 5, I test a normal satellite image. The CNN correctly predicts PermanentCrop with 99.2% confidence."

Cell 6: Demo - Edge Cases & Noise

What it does: Tests invalid text file and pure white noise.

Say: "In Cell 6, I test edge cases. The text file is safely caught with an error message. But for random white noise, the network predicts Highway with 88.8% confidence. This proves an important finding: Softmax neural networks can be overly confident on out-of-distribution inputs."